

TMC-104 – ADVANCE OPERATING SYSTEM

MCA First Semester — Mid Term (Odd) Semester Examination, October 2024

Graphic Era Hill University | Complete Solved Answer Key with Detailed Explanations

Reference: *Operating System Concepts*, 9th Edition — Abraham Silberschatz, Peter B. Galvin, Greg Gagne

Note: The original paper requires attempting **one sub-question [(a) OR (b)]** from each of the 5 questions, 10 marks each (Total = 50 marks). For complete revision of the syllabus, **both alternatives of every question are solved in full below.**

Q1. (a) Define an operating system and provide an explanation of the layered architecture of an operating system.

[CO1] Marks: 10

Definition

An Operating System (OS) is system software that acts as an intermediary between the user/application programs and the computer hardware. It manages hardware resources such as the CPU, memory and I/O devices, and provides a convenient, efficient and secure environment for executing programs. Silberschatz describes it as both a "resource allocator", managing and allocating resources among competing requests, and a "control program" that controls execution of user programs to prevent errors and improper use of the system.

Layered Approach (Layered Architecture)

The layered approach is a structuring technique in which the operating system is broken into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0) is the hardware, and the highest layer (layer N) is the user interface. Each layer uses only the functions and services of the layers below it, hiding its own internal implementation from the layers above – this is a classic example of information hiding / modularity, first demonstrated in Dijkstra's THE operating system.

Layered Architecture of an Operating System (THE OS model)

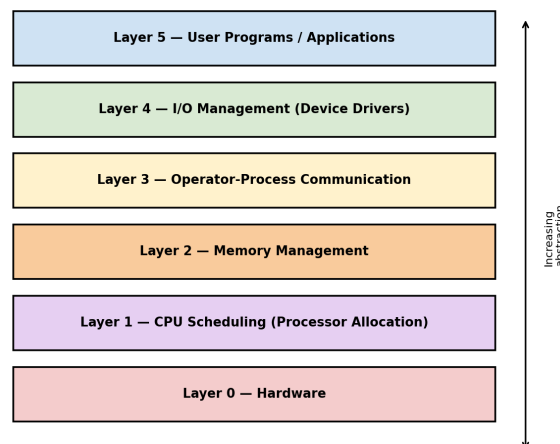


Fig. 1: Layered Architecture of an Operating System

Typical layer breakdown (illustrative, as in the THE OS)

- **Layer 0 – Hardware:** the physical CPU, memory and I/O devices.
- **Layer 1 – CPU Scheduling:** allocates the processor among competing processes.

- **Layer 2 – Memory Management:** allocates space for processes in main memory (and on the drum/disk in the original THE system).
- **Layer 3 – Operator-Process Communication:** handles console/operator messages to and from each process.
- **Layer 4 – I/O Management:** buffers information transferred to/from peripheral devices, via device drivers.
- **Layer 5 – User Programs / Applications:** user processes needing no further knowledge of process, memory, or device management.

Advantages

- Simplifies debugging and system verification – each layer can be tested and verified independently, since it uses only already-verified, lower-level services.
- Modularity – changes to the implementation of one layer do not affect other layers, as long as the interface between them stays the same.

Disadvantages

- Careful definition of layers is required because a layer can use only lower layers, which can be restrictive (e.g., I/O may need to occur at multiple different layers).
- Each layer adds overhead to system calls, so a layered system tends to be less efficient than a monolithic system where cross-layer calls are avoided.

Q1. (b) Write a short note on the following: (i) Directory and Sub-directory (ii) Boot block (iii) Inode block (iv) Super block (v) Group descriptor

[CO1] Marks: 10 (2 marks each)

Typical UNIX/ext2 File-System Layout on Disk

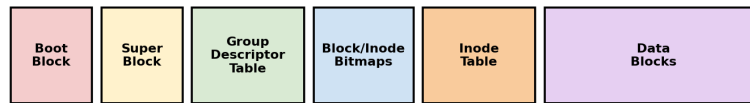


Fig. 2: A typical UNIX/ext2-style file-system layout on disk

(i) Directory and Sub-directory

A **directory** is a special file (or file-system object) that contains information – name, attributes/metadata and location – about a group of other files, and provides the mapping used to locate them on the disk. A **sub-directory** is simply a directory contained within another directory, allowing directories to be organised as a tree structure so that users can group related files hierarchically (e.g., /home/user1/documents). This hierarchical structuring makes it possible to have files with the same name as long as they lie in different directories.

(ii) Boot Block

The **boot block** is the very first block(s) of a disk/partition, reserved to hold a small bootstrap program (bootstrap loader) that is read into memory by the computer's ROM code at startup and used to load the operating system kernel into memory and start it running. A disk that contains at least one partition with a boot block is called a *boot disk* or *system disk*.

(iii) Inode Block

In UNIX-style file systems, every file is represented internally by an **index-node (inode)**, a data structure that stores all the metadata about the file – owner, permissions, size, timestamps, and pointers to the data blocks (or indirect blocks for large files) that hold the file's actual content – but not the file's name (which is kept in the directory entry). The **inode table/block** is the disk area that stores these inode structures for all files in the file system.

(iv) Super Block

The **super block** is a control block located near the start of a file system/partition that stores global metadata about the file system as a whole – e.g., total number of blocks and inodes, free block/inode counts, block size, and the file system's state (clean/dirty). Because it is critical, the OS usually keeps backup copies of the super block at fixed intervals within the file system to allow recovery if it becomes corrupted.

(v) Group Descriptor

In file systems such as ext2/ext3/ext4, the disk is divided into **block groups** to keep related inodes and data blocks physically close (reducing seek time). Each block group has a corresponding entry in the **Group Descriptor Table**, which records information such as the location of that group's block bitmap, inode bitmap, inode table, and the count of free blocks/inodes/directories within the group – this allows the file system to quickly locate free space and metadata for each group.

Directory and Sub-directory Tree Structure

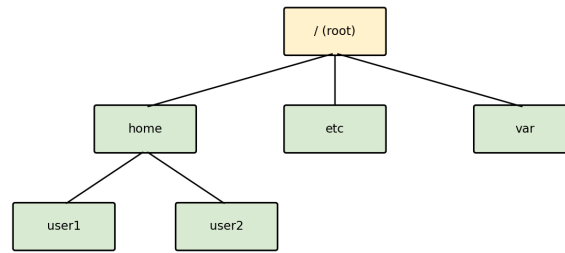


Fig. 3: Directory and sub-directory tree structure

Q2. (a) Explain the various file systems used in different operating systems. Compare and contrast sequential, direct, and indexed file organization methods.

[CO1] Marks: 10

File Systems Used in Different Operating Systems

OS	File System(s)	Key Feature
Windows	FAT32, exFAT, NTFS	NTFS supports journaling, permissions (ACLs), encryption and large file/volume support.
Linux/UNIX	ext2, ext3, ext4, XFS, Btrfs	ext3/ext4 add journaling for crash recovery; use inodes and block groups.
macOS	APFS (formerly HFS+)	APFS is optimized for SSDs, supports snapshots and strong encryption.
Optical/removable media	ISO 9660, UDF	Designed for read-only or write-once optical media.

Regardless of the underlying disk format, every file system must provide mechanisms for naming files, organising them into directories, allocating disk space to them, and controlling access – the specific data structures (FAT tables, inodes, B-trees, etc.) differ, but the goals are the same.

File Access / Organization Methods

When information stored in a file is to be used, it must be accessed and read into computer memory. There are several ways to structure this access (Silberschatz, Ch.11):

Basis	Sequential Access	Direct (Random) Access	Indexed Access
Concept	Records/bytes are read or written in order, one after another, starting from the beginning of the file.	File is viewed as a random access storage; any record can be accessed directly by its address.	Records are stored in blocks, and each block has a pointer to its location in the file.
Analogy	Reading a cassette tape from start to finish.	Retrieving tracks on a CD/DVD without playing the rest of the disc.	Looking up a word using a dictionary index / library catalog.
Speed for random lookup	Slow – must read through all previous records.	Fast – direct jumping to the record.	Fast – direct jumping to the record via index.
Best suited for	Applications that process most of the data sequentially.	Applications needing immediate access to specific records.	Applications needing fast access to specific records.
Implementation basis	Simple sequential/linked or chained list.	Requires a fixed record size and a table of pointers.	Requires an index table and a pointer to the first record.

In practice, indexed access can be viewed as an enhancement over direct access: instead of the application needing to know a record's exact block number, the index translates a meaningful key into that block number, combining the ordering convenience of sequential files with much of the speed of direct-access files.

Q2. (b) Explain the access permissions available in LINUX. What do you understand by access matrix?

[CO1] Marks: 10

Access Permissions in LINUX

Linux (a UNIX-like system) controls access to every file and directory using a permission model defined for three classes of users and three types of permissions.

Class		Meaning	
Owner (u)		The user who owns the file.	
Group (g)		Users belonging to the file's group.	
Others (o)		All other users on the system.	

Permission	Symbol	Effect on a File	Effect on a Directory
Read	r	View the contents of the file.	List the contents (file names) of the directory.
Write	w	Modify or overwrite the contents of the file.	Create, delete or rename files within the directory.
Execute	x	Run the file as a program/script.	Enter (cd into) the directory / access its files.

Permissions are displayed/set as a 10-character string, e.g. **-rwxr-xr--**: the first character denotes the file type (- for regular file, d for directory), followed by three permission triplets for owner, group and others respectively. They can also be represented numerically (octal), where r=4, w=2, x=1 are summed per class – e.g. **chmod 754 file** sets rwx for owner, r-x for group and r-- for others. Commands such as **chmod** (change mode), **chown** (change owner) and **chgrp** (change group) are used to manage these permissions.

Access Matrix

The access matrix is a conceptual model used to describe the protection state of a system (Silberschatz, Ch.14). The rows of the matrix represent **domains** (a domain defines the set of objects and the operations that can be invoked on each object – often corresponding to a user or a process), and the columns represent **objects** (files, devices, memory segments, etc.) in the system. Each entry $access(i, j)$ lists the set of operations (read, write, execute, etc.) that a process executing in domain D_i can perform on object O_j .

Domain \ Object	File F1	File F2	Printer P1
D1	read	read, write	-
D2	execute	-	print
D3	read, write	read	print

The access matrix is a useful abstraction because it precisely defines the protection state of the system at any moment, and it can be dynamically modified (entries can change as domain switching or the granting/revoking of rights occurs). In practice the matrix is usually sparse, so operating systems implement it efficiently either by storing it column-wise (an **Access Control List**, ACL – per object, listing which domains/users have which rights) or row-wise (a **capability list** – per domain, listing which objects it can access and with what rights). Linux's owner/group/other permission bits are, in effect, a compact, simplified implementation of an access-control-list style access matrix.

Q3. (a) Differentiate between the following: (i) System Calls vs Functions (ii) Windows vs Linux

[CO1] Marks: 10 (5 marks each)

(i) System Calls vs Functions

Basis	System Call	Function (Library/User function)
Definition	A request made by a user program to the OS kernel to perform a privileged operation.	A self-contained block of code performing a specific task, written in a higher-level language.
Execution mode	Causes a switch from user mode to kernel mode.	Executes entirely in user mode within the calling process's context.
Provider	Provided by the operating system kernel.	Provided by the programmer, or a user-level library (e.g., C standard library).
Overhead	Higher – involves mode switching and context switching.	Lower – simple procedure call/return.
Example	open(), fork(), read(), write(), exit().	A user-defined function add(a,b), or printf() (which internally uses system calls).

(ii) Windows vs Linux

Basis	Windows	Linux
Source model	Proprietary/closed-source (owned by Microsoft).	Open-source (kernel developed collaboratively; distributions made by various groups).
Kernel type	Hybrid kernel (Windows NT kernel).	Monolithic kernel (with loadable modules).
Cost	Usually paid, licensed per device.	Mostly free, though paid enterprise support (e.g., RHEL) exists.
File system	NTFS / exFAT / FAT32.	ext2/ext3/ext4, XFS, Btrfs, etc.
Case sensitivity	File names are generally case-insensitive.	File names are case-sensitive.
User interface	GUI-centric, with command-line (PowerShell/CMD) as a secondary interface.	Traditionally CLI-centric, though modern distros offer full GUIs.
Security model	User Account Control (UAC), ACL-based permissions.	Capabilities/Group/Other permission bits, SELinux/AppArmor for mandatory access control.
Typical use	Desktop PCs, gaming, enterprise office environments.	Servers, embedded systems, supercomputers, developer workstations.

Q3. (b) Explain the different types of attributes and operations related to file.

[CO1] Marks: 10

File Attributes

A file is named, for the convenience of its human users, and is referred to by that name. Besides a name, a file has a number of other attributes that vary from one OS to another, but typically include (Silberschatz, Ch.11):

- **Name** – the symbolic file name, the only information kept in human-readable form.
- **Identifier** – a unique tag (usually a number) that identifies the file within the file system; the non-human-readable name for the file.
- **Type** – needed for systems that support different file types (e.g., .txt, .exe, .jpg).
- **Location** – a pointer to a device and to the location of the file on that device.
- **Size** – the current size of the file (in bytes/words/blocks), and possibly the maximum allowed size.
- **Protection** – access-control information determining who can read, write or execute the file.
- **Time, date, and user identification** – data kept for creation, last modification, and last use; useful for protection, security and usage monitoring.

This information is stored in the directory structure/inode, which resides on secondary storage.

File Operations

A file is an abstract data type, and the OS provides several basic operations to manipulate it:

- **Creating a file** – space for the file is found in the file system and a new entry is made in the directory.
- **Writing a file** – a system call specifying the file name and the information to write; the system keeps a write pointer to the location where the next write is to take place.
- **Reading a file** – a system call specifying the file name and where the next block should be put; the system keeps a read pointer to the location of the next read.
- **Repositioning within a file (seek)** – the directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value, without any actual I/O taking place.
- **Deleting a file** – the directory is searched for the named file; once found, the associated file space and directory entry are released.
- **Truncating a file** – the user erases the contents of a file but keeps its attributes, resetting the file length to zero and releasing its space, without having to delete and recreate the file.
- **Opening/Closing a file** – most systems require a file to be *opened* (using an `open()` system call) before use, which brings its attributes and disk-block location into main memory for fast subsequent access, and *closed* after use to release these in-memory resources and flush any buffered writes to disk.

Together, these operations, along with attributes like protection information, allow the OS to present a uniform, convenient abstraction of secondary-storage data as named files, regardless of the physical storage medium or file-system implementation underneath.

Q4. (a) Explain the different types of real time operating system. Also explain the different characteristics of RTOS.

[CO2] Marks: 10

What is a Real-Time Operating System?

A Real-Time Operating System (RTOS) is an OS designed to serve real-time applications, where correctness of a computation depends not only on its logical result but also on the time at which the result is produced. Tasks must be completed within specified time constraints (deadlines).

Types of Real-Time Operating Systems

1. Hard Real-Time Systems

Guarantee that critical tasks are completed within their deadline; missing even a single deadline is considered a total system failure. Such systems have limited utility beyond their few dedicated functions.

Example: flight control systems, airbag deployment systems, nuclear reactor control.

2. Soft Real-Time Systems

A critical real-time task receives priority over other tasks and retains that priority until it completes, but an occasional missed deadline is tolerable and merely results in degraded (not catastrophic) performance.

Example: multimedia streaming, online transaction systems, digital audio/video playback.

3. Firm Real-Time Systems

Occasional deadline misses are tolerated, but if a task result arrives late it is considered useless (though not catastrophic) – a small number of missed deadlines is acceptable but excessive misses degrade quality unacceptably. **Example:** certain industrial automation and robotics tasks.

Characteristics of an RTOS

- **Determinism** – operations are performed at fixed, predictable points in time or within predictable time intervals.
- **Responsiveness** – the time taken to acknowledge an interrupt and begin its service must be small and predictable.
- **User control** – the user/programmer can typically specify task priorities, and control which tasks may be preempted.
- **Reliability** – the system must continue to operate correctly even under transient overloads or partial failures; degradation should be graceful, not catastrophic.
- **Fail-soft operation** – the ability of a system to fail in a way that preserves as much capability and data as possible, ensuring critical functions continue.
- **Priority-based preemptive scheduling** – higher-priority (more time-critical) tasks can preempt lower-priority tasks so critical deadlines are met.
- **Minimal interrupt latency and context-switch time** – the kernel must respond and switch tasks with very small and bounded overhead.

Q4. (b) Consider a real-time system with three tasks, each with its execution time (C), period (T). The tasks are scheduled using the Rate Monotonic Scheduling algorithm. Check whether the tasks can be scheduled or not.

[CO2] Marks: 10

Task	Execution Time (C)	Period (T)
A	2 ms	8 ms
B	3 ms	6 ms
C	1 ms	12 ms

Concept: In Rate Monotonic Scheduling (RMS), tasks are assigned fixed priorities based on their period – the *shorter* the period, the *higher* the priority. Deadlines are assumed equal to periods.

Step 1: Assign priorities (shortest period = highest priority)

Task B (T=6 ms) > Task A (T=8 ms) > Task C (T=12 ms)

Step 2: Liu & Layland utilization bound test (sufficient condition)

Total CPU utilization: $U = C_A/T_A + C_B/T_B + C_C/T_C = 2/8 + 3/6 + 1/12 = 0.25 + 0.5 + 0.0833 = \mathbf{0.8333}$

For $n = 3$ tasks, the Liu-Layland least upper bound is $U_{\text{bound}} = n(2^{1/n} - 1) = 3 \times (2^{1/3} - 1) \approx 3 \times 0.2599 = \mathbf{0.7798}$

Since $U = 0.8333 > U_{\text{bound}} = 0.7798$, the task set **fails the sufficient utilization-bound test**. However, this only means schedulability is *not guaranteed* by this simple sufficient test – it does *not* prove the task set is unschedulable, since $U < 1$. An exact test (Response Time Analysis) is needed to determine the true outcome.

Step 3: Exact test – Response Time Analysis (RTA)

For each task, the worst-case response time R is computed iteratively as $R^{(k+1)} = C_i + \sum_{j \in \text{hp}(i)} \lceil R^{(k)} / T_j \rceil C_j$, starting with $R^{(0)} = C_i$, where $\text{hp}(i)$ is the set of tasks with higher priority than i .

Task B (highest priority, no interference): $R_B = C_B = 3 \text{ ms} \leq T_B = 6 \text{ ms} \rightarrow \mathbf{\text{meets deadline}}$

Task A (interfered by B):

- $R^{(0)} = 2$
- $R^{(1)} = 2 + \lceil 2/6 \rceil \times 3 = 2 + 3 = 5$
- $R^{(2)} = 2 + \lceil 5/6 \rceil \times 3 = 2 + 3 = 5$ (converged)

$R_A = 5 \text{ ms} \leq T_A = 8 \text{ ms} \rightarrow \mathbf{\text{meets deadline}}$

Task C (interfered by both B and A):

- $R^{(0)} = 1$
- $R^{(1)} = 1 + \lceil 1/6 \rceil \times 3 + \lceil 1/8 \rceil \times 2 = 1 + 3 + 2 = 6$
- $R^{(2)} = 1 + \lceil 6/6 \rceil \times 3 + \lceil 6/8 \rceil \times 2 = 1 + 3 + 2 = 6$ (converged)

$R_C = 6 \text{ ms} \leq T_C = 12 \text{ ms} \rightarrow \mathbf{\text{meets deadline}}$

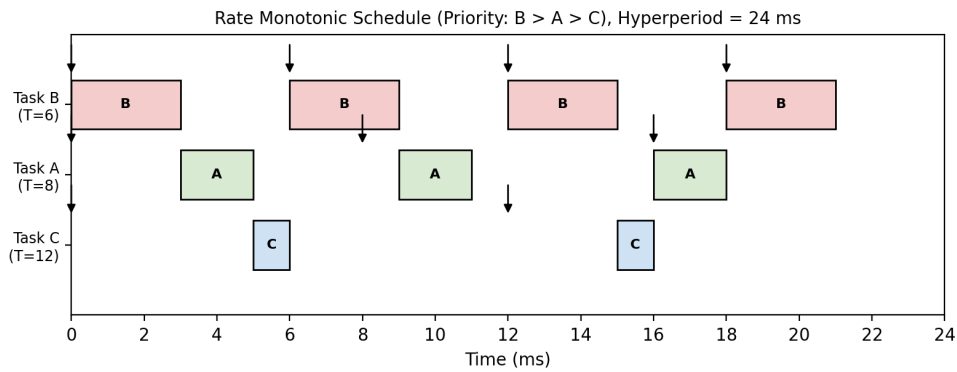


Fig. 4: RMS schedule over one hyperperiod (LCM of 6, 8, 12 = 24 ms) – confirms every job instance completes before its deadline

Conclusion

Although the task set fails the simple Liu-Layland sufficient utilization-bound test, the exact Response Time Analysis (confirmed by the timeline simulation above) shows that every task's worst-case response time is within its period. Therefore, **the task set IS schedulable** under Rate Monotonic Scheduling. This illustrates an important concept: the utilization bound test is only *sufficient*, not *necessary* – failing it does not automatically imply the task set is infeasible; an exact test must be used to be certain.

Q5. (a) A disk has 200 cylinders, numbered from 0 to 199. The disk arm is currently at cylinder 100. The queue of pending disk requests contains the following cylinder numbers: 37, 83, 27, 122, 18, 124. Calculate the total head movement using FCFS and SSTF disk scheduling algorithms.

[CO2] Marks: 10

Given: Cylinders 0–199, current head position = 100, request queue = 37, 83, 27, 122, 18, 124 (in that order of arrival).

(1) First-Come-First-Served (FCFS)

In FCFS, requests are serviced strictly in the order they arrive in the queue, regardless of their location on the disk.

Move	From	To	Head Movement
1	100	37	$ 100-37 = 63$
2	37	83	$ 37-83 = 46$
3	83	27	$ 83-27 = 56$
4	27	122	$ 27-122 = 95$
5	122	18	$ 122-18 = 104$
6	18	124	$ 18-124 = 106$

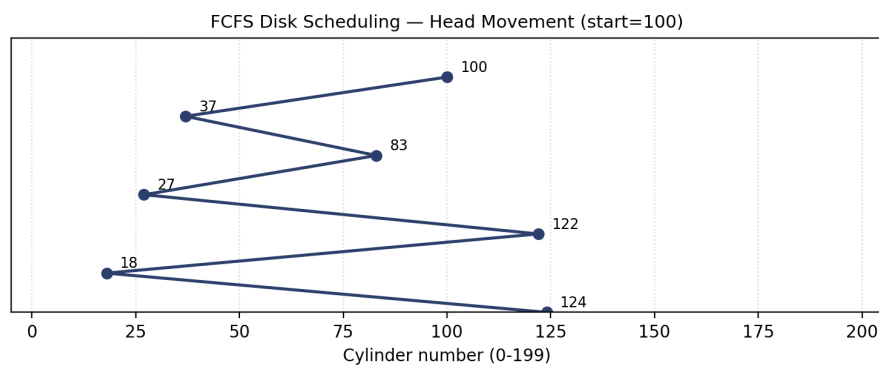


Fig. 5: Head movement path – FCFS

Total head movement (FCFS) = $63+46+56+95+104+106 = 470$ cylinders

(2) Shortest Seek Time First (SSTF)

In SSTF, at every step the request whose cylinder is *closest* to the current head position is serviced next.

Move	From	To	Head Movement	Reasoning (nearest of remaining)
1	100	83	17	$ 100-83 =17$ is smallest among 37,83,27,122,18,124
2	83	122	39	$ 83-122 =39$ is smallest among 37,27,122,18,124
3	122	124	2	$ 122-124 =2$ is smallest among 37,27,18,124
4	124	37	87	$ 124-37 =87$ is smallest among 37,27,18
5	37	27	10	$ 37-27 =10$ is smallest among 27,18
6	27	18	9	only 18 remains

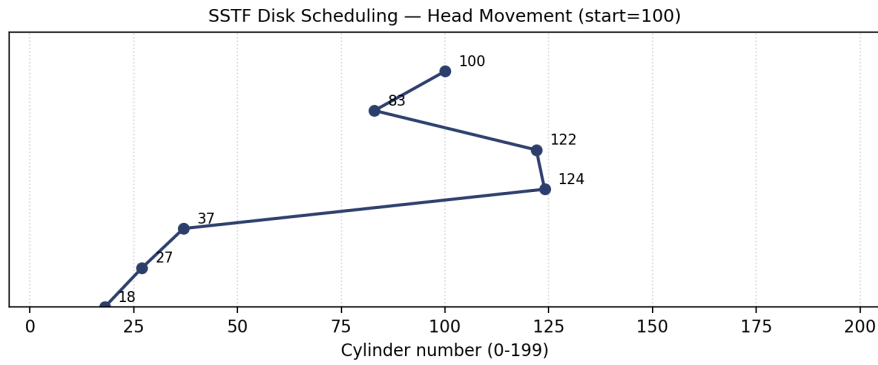


Fig. 6: Head movement path – SSTF

Total head movement (SSTF) = $17+39+2+87+10+9 = 164$ cylinders

Comparison

SSTF (164 cylinders) results in significantly less total head movement than FCFS (470 cylinders) for this request set, since SSTF always greedily chooses the nearest pending request, reducing overall seek distance and hence average seek time. However, SSTF can potentially cause *starvation* of requests that are far from the current head position if closer requests keep arriving, whereas FCFS is inherently fair (though it may cause more overall arm movement).

Q5. (b) Write a short note on (i) Data Protection and Security goals of operating system (ii) Access permissions of files and directories.

[CO2] Marks: 10 (5 marks each)

(i) Data Protection and Security Goals of an Operating System

Protection refers to mechanisms for controlling the access of programs, processes, or users to system resources; security refers to defending the system from both internal and external, malicious or accidental attacks. Together, they aim to achieve the following core goals (often summarised as the **CIA triad**):

- **Confidentiality** – ensuring information is accessible only to those authorized to access it, preventing unauthorized disclosure (e.g., via encryption, access controls).
- **Integrity** – ensuring data is not altered or destroyed in an unauthorized manner, so information remains accurate and trustworthy.
- **Availability** – ensuring authorized users have reliable and timely access to resources and information when needed, guarding against denial-of-service disruptions.
- **Authentication** – verifying the identity of a user or process (passwords, biometrics, tokens) before granting access.
- **Authorization** – ensuring an authenticated user/process can perform only the operations it has explicitly been granted rights to (via access-control lists, capabilities, or the access matrix model).
- **Accountability (Auditing)** – maintaining logs of system activity so actions can be traced back to the responsible user/process, aiding detection and investigation of breaches.

To achieve these goals, an OS employs mechanisms such as user authentication, file/directory permission bits, access control lists, encryption, firewalls, intrusion detection, and regular security patching/updates.

(ii) Access Permissions of Files and Directories

Most modern operating systems (notably UNIX/Linux) protect files and directories using a permission scheme based on three user classes – **owner**, **group** and **others** – and three permission types:

Permission	On a File	On a Directory
Read (r)	View file contents.	List files inside the directory.
Write (w)	Modify/overwrite file contents.	Create, delete, rename files within the directory.
Execute (x)	Run the file as a program/script.	Enter (cd) the directory and access files by name.

Permissions are commonly displayed in the form **-rwxr-xr--** (type + owner/group/other triplets) or as a three-digit octal number (e.g., 754), and are changed using commands like *chmod*, with ownership changed via *chown/chgrp*. Some file systems (e.g., NTFS) extend this with full Access Control Lists (ACLs), allowing fine-grained permissions to be assigned to individual users or groups beyond the simple owner/group/other model – conceptually, both are practical, compact implementations of the theoretical access matrix model.

Prepared for MCA First Semester revision. Primary reference: Operating System Concepts, 9th Edition by Abraham Silberschatz, Peter B. Galvin and Greg Gagne (Wiley). Diagrams are original illustrations created for this answer key based on standard OS textbook concepts.